Day - 06 Theory Assignment

Problem 1: The Jacobin and the chain Rule

In DNNs, data passes through multiple sequential layer. Suppose layer 1 is a function $g(x)$ that maps a vector to a vector, and layer 2 is a function $f(g(x))$ that maps the intermadiate vector to the final o/p vector.

Task A: If you find the derivatives of the final output you must use multivariable chain rules. mathmatically explain how the jacobi matrices of f, g are combined to find this final derivative.

Task B: Why is it impossible to represent the derivative of a multioutput NN layer using a single standard gradient vector? explain what the rows and cols of the jacobin specially represent.

CamScanner

Solution:

Task A: According the multivariable chain Rule, the derivative of the composit function $f(g(x))$ is calculated By performing matrix multiplication on their ~~repre~~ respective Jacobin matrices.

Specially: $J_{final} = J_f \times J_g$

Task B:

A. Standard Gradient vector only works for a fn that outputs a single scalar number. A NN, output a vector of many numbers. So we must use Jacobian matrix.

The row represent the o/p variable and the col represent the i/p variable.

problem 2: The hessian and saddle points.

TASK A: State clairauts Thm and explain why it guarantees that the hessian matrix of a continously differentiable loss function is always perfectly symmetric.

TASK B: Geometrically, what does it mean if the hessian matrix evaluated at a critical point has both positive and Negative eigenvalues? Why does this specific geometry cause standard Gradient Descent algorithm to stall or significantly slow down?

Solution:

TASK A: Clairauts Theorem guarantees that for continously differentiable function. the order of taking partial derivatives doesn't matter. $\frac{\partial^2 L}{\partial x \partial y} = \frac{\partial^2 L}{\partial y \partial x}$

Hessian matrix is perfectly symmetric $(H = H^T)$

CamScanner

Task B:

Mixed positive & Negative eigenvalues define an indefinite matrix meaning you are at saddle point. Geometrically the landscap curve up in one direction and down in Another direction.

The gradient Decent stalls here becase the gradient approaches is zero.

Problem 3: The curse of second order optimizer

First order optimization algorithm (like gd Descent) only use the gradient (∇L) to update weights.

Second order optimization algorithm (like Newton method) use Both Gradient and the inverse of the hessian matrix (H^{-1}).

The weight update rule for Newton's method is

$$W_{new} = W_{old} - H^{-1} \cdot \nabla L(W_{old}).$$

<u>Task A:</u> Geometrically, why does multiplying the gradient by the inverse hessian matrix allow Newton's method to take massive, highly accurate steps directly to the minimum often converging in just one two steps?

<u>Task B:</u> If Newton's method is mathmatically vastly superior to Gradient Descent. Why is Never used to train modern DNNs with million or billion parameters N.

<u>Solution:</u>

<u>Task A:</u> Multiplying the inverse of Hessian (H^{-1}) divides the gradient by the curvature. It normalize the landscape, flating out steep ravines and allowing

the optimizer to point directly at the geometric center of the minimum, according for all curvature distortions.

<u>Task B:</u> A NN, with N parameters would have NXN Hessian matrix. Calci the inverse of a matrix take roughly $O(N^3)$ time complexity. for Billion parameter its impossible for Store data.